

ADR-003 — KrakenD comme API Gateway (vs Spring Cloud Gateway)

Champ	Valeur
Statut	Accepté
Date	2026-01-22
Auteurs	Équipe CanBankX

Contexte

Un API Gateway est requis pour :

- Exposer un point d'entrée unique (`localhost:8080`) aux clients
- Router les requêtes vers les bons microservices
- Agréger des réponses multi-services (ex : résumé de compte = données `account-service` + `payment-service`)
- Filtrer/propager les en-têtes (`Authorization`, `Idempotency-Key`, `Content-Type`)
- Permettre l'équilibrage de charge entre plusieurs instances

Les options considérées :

1. **Spring Cloud Gateway** (code Java) — configurable en code, familier pour les développeurs Spring.
2. **KrakenD** (binaire Go, config JSON déclarative) — ultra-performant, zéro code, agrégation native.
3. **NGINX** — proxy inverse simple mais sans agrégation ni logique métier.
4. **Kong** — complet mais complexe à opérer.

Décision

KrakenD 2.7 est choisi comme API Gateway.

La configuration est entièrement déclarative dans `krakend/krakend.json`. Le routage suit le pattern :

```
Client → KrakenD :8080 → /api/{resource}
      → identity-service :8081 /identityservice/...
      → account-service  :8082 /accountservice/...
      → payment-service  :8083 /paymentservice/...
```

L'endpoint d'agrégation `/api/accounts/{id}/summary` fusionne en parallèle les réponses de `account-service` et `payment-service` en une seule réponse JSON avec groupes nommés (`account`, `recentTransactions`).

L'équilibrage de charge est configuré au niveau des backends `payment-service` en fournissant plusieurs `host` :

```
"host": ["http://payment-service:8083", "http://payment-service-2:8083"]
```

Conséquences

Positives

- **Aucun code applicatif** : le gateway est entièrement défini par un fichier JSON, sans risque d'introduire des bugs dans le chemin critique.
- **Agrégation native** : la fusion **account + recentTransactions** est triviale avec KrakenD, elle aurait nécessité un service orchestrateur avec Spring Cloud Gateway.
- **Performance** : KrakenD est écrit en Go et introduit une surcharge négligeable (confirmé par les tests k6 : latence gateway < 2 ms en steady-state).
- **Load balancing round-robin** : KrakenD distribue les requêtes entre instances sans configuration supplémentaire.

Négatives / Risques

- **Moins flexible** : certaines logiques complexes (ex : authentification JWT avec validation de claims) nécessiteraient un plugin KrakenD ou un service intermédiaire.
- **Pas de circuit breaker natif** dans cette configuration : une instance défaillante reste dans la rotation jusqu'à ce que KrakenD détecte le timeout (5 s).
- **Observabilité limitée du gateway** : KrakenD n'expose pas de métriques Prometheus dans cette configuration ; l'observabilité est assurée côté services (Spring Actuator).